EduCore — Technical Documentation

System architecture, database schema, and API/route reference for the EduCore platform.

- **Live app:** http://educoreasp.runasp.net/
- **Stack:** ASP.NET Core MVC (.NET 10), Entity Framework Core 10, SQL Server

---

1. System Architecture

EduCore is a **server-rendered ASP.NET Core MVC** web application following a classic layered design.

```
Browser (Razor views + Bootstrap/custom CSS)
        │  HTTP(S)
        ▼
ASP.NET Core MVC  ──►  Controllers  ──►  EF Core (AppDbContext)  ──►  SQL Server
        │                    │
   Cookie auth        ViewModels / Models
   middleware
```

Request flow

1. A request hits the ASP.NET Core middleware pipeline (HTTPS redirect → routing → **authentication** → **authorization**).
2. Routing maps `/{Controller}/{Action}/{id?}` to a controller action.
3. `[Authorize(Roles = "...")]` gates teacher-only / student-only areas.
4. The controller uses `AppDbContext` (EF Core) to read/write **SQL Server**, builds a model/view model, and returns a Razor view.
5. Razor renders HTML using the shared layout (`_EduCoreLayout`) and `wwwroot/css/shared.css` .

Key cross-cutting concerns

- **Authentication:** custom cookie auth against the `Teachers` / `Students` tables (no ASP.NET Identity). Claims carry the user id, name, and role.
- **Authorization:** role-based (`Teacher` / `Student`) via `[Authorize]` .
- **Culture:** currency formatted as EGP globally (set in `Program.cs`).
- **Migrations on startup:** `Program.cs` calls `db.Database.Migrate()` at boot so the deployed app creates/updates its own schema (important on MonsterASP, whose DB is only reachable from inside the host).

Deployment

- Hosted on **MonsterASP.NET** (ASP.NET Core + SQL Server).
- Configuration (connection string) via `appsettings.json` / host environment variables.
- The database schema is created automatically on first startup via EF Core migrations.

---

2. Technology Stack

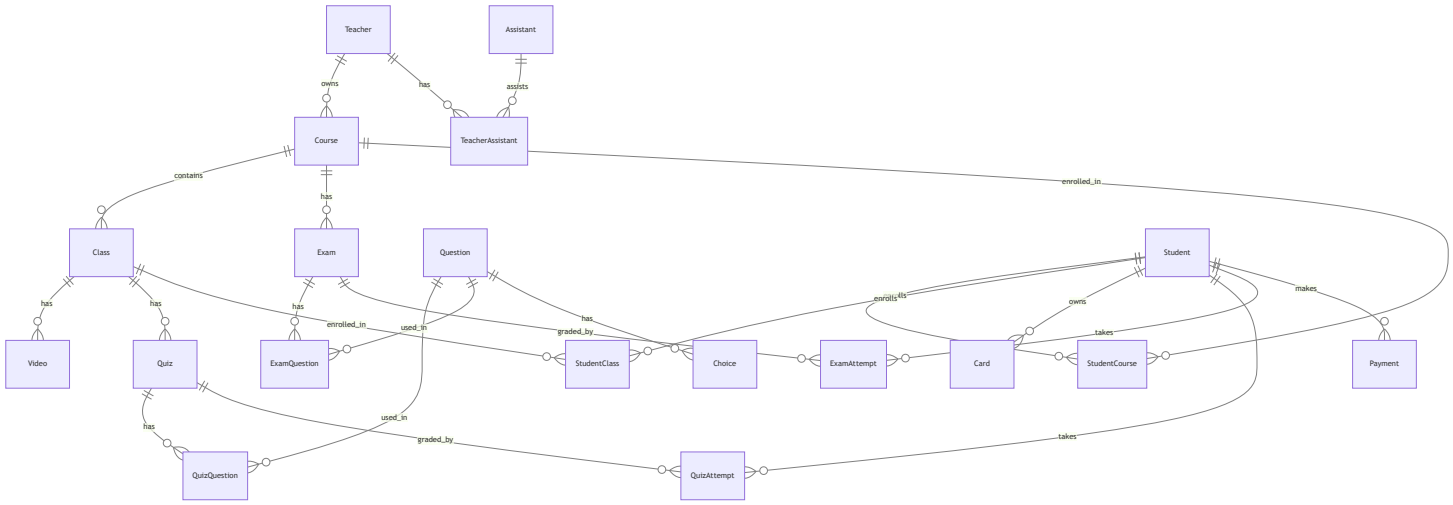
Layer	Technology
Runtime	.NET 10
Web framework	ASP.NET Core MVC
ORM	Entity Framework Core 10 (code-first migrations)
Database	Microsoft SQL Server
Auth	Cookie authentication + PBKDF2 (SHA-256) password hashing
Views	Razor + Bootstrap-based custom CSS
Hosting	MonsterASP.NET

3. Project Structure

```
EduCore/EduCore/            # ASP.NET Core MVC project
├─ Controllers/             # Account, Home, Dashboard, Courses, Classes, Videos,
│                           #   Quizzes, Questions, Exams, ExamQuestions, Catalog,
│                           #   Learn, Assessments, Payment, Cards, Results, Revenue, Profile
├─ Models/                  # EF entities (domain model)
├─ ViewModels/              # form & display view models
├─ Views/                   # Razor views per controller + shared layouts
├─ Data/AppDbContext.cs     # EF Core DbContext + Fluent config + seed
├─ Helpers/                 # PasswordHasher, ClaimsPrincipalExtensions,
│                           #   VideoEmbedHelper, PlatformSettings
├─ Migrations/              # EF Core migrations + model snapshot
├─ wwwroot/                 # static assets (css/js/lib)
├─ appsettings.json         # connection string & config
└─ Program.cs               # startup: DI, auth, culture, startup migration
```

4. Database Schema

Entity–relationship diagram



Tables

Table	Key columns	Purpose
Teacher	ID, FName, LName, Email (unique), Password, PhoneNumber, Biography	Teacher accounts
Student	ID, FName, LName, Email (unique), Password, PhoneNumber	Student accounts
Assistant	ID, FName, LName, Email, Password, PhoneNumber, Biography	Assistant accounts (schema only)
TeacherAssistant	ID, TeacherID, AssistantID	Teacher ↔ Assistant link
Course	ID, Name, Price, Enrollable, AllowCourseEnrollment, TeacherID	A course owned by a teacher
Class	ID, Name, Price, Enrollable, PDF, HomeworkPDF, CourseID	A class within a course
Video	ID, Title, URL, ClassID	Lecture video (YouTube/Vimeo/mp4 URL)
Quiz	ID, Title, DurationMinutes, ClassID	Class quiz (timed)
Exam	ID, Title, DurationMinutes, CourseID	Course exam (timed)
Question	ID, QuestionText	A multiple-choice question
Choice	ID, Text, IsCorrect, QuestionID	One answer choice (normalized)
QuizQuestion	ID, QuizID, QuestionID	Quiz ↔ Question link
ExamQuestion	ID, ExamID, QuestionID	Exam ↔ Question link
StudentCourse	ID, StudentID, CourseID	Course enrollment
StudentClass	ID, StudentID, ClassID	Individual class enrollment (à la carte)
QuizAttempt	ID, StudentID, QuizID, Score, TotalQuestions, SubmittedAt	Saved quiz result
ExamAttempt	ID, StudentID, ExamID, Score, TotalQuestions, SubmittedAt	Saved exam result
Card	ID, CardholderName, CardNumber, ExpiryDate, CVV, StudentID	Saved payment card (simulated)
Payment	ID, StudentID, TeacherID, ItemType, ItemName, Amount, PaidAt, CardLast4	Purchase record (snapshot)

Notable schema decisions

- **Normalized choices:** `Question` has many `Choice` rows (each with `IsCorrect`) instead of storing choices in a single string — cleaner grading and querying.
- **Payment snapshots:** `Payment` stores `ItemType` / `ItemName` / `TeacherID` as a snapshot (no hard FK to Course/Class), so purchase history and teacher revenue survive course/class deletion.
- **Cascade rules:** junction/attempt tables have two foreign keys; one side is set to `Restrict` (in `OnModelCreating`) to avoid SQL Server's "multiple cascade paths" error. Owner→child links (Course→Class→Video/Quiz, Question→Choice) cascade.
- **Unique email** per role table (`Teachers`, `Students`), enforced by a unique index.

---

5. Authentication & Authorization

- **Passwords** are hashed with **PBKDF2 (SHA-256, 100k iterations, per-user salt)** — see `Helpers/PasswordHasher.cs`. Stored as `base64(salt):base64(hash)`.

- **Login** looks up the email in the chosen role's table, verifies the hash, and issues an **encrypted, signed cookie** containing claims: `NameIdentifier` (user id), `Name`, `Role`.
- **On each request**, the cookie middleware rebuilds the user into `HttpContext.User`. `[Authorize(Roles="Teacher"|"Student")]` gates access; `User.GetUserId()` returns the current user's id.
- **Roles are implicit** from which table the user is in — no separate role table.

---

6. API / Routes Reference

EduCore is a **server-rendered MVC app**, not a JSON/REST API. Routes follow `/{Controller}/{Action}/{id?}`.

Account (anonymous)

Method	Route	Purpose
GET	`/`	Redirects to Login (or the user's home if signed in)
GET/POST	`/Account/Login`	Sign in (role-based)
GET/POST	`/Account/Register`	Sign up (Student or Teacher)
POST	`/Account/Logout`	Sign out

Teacher (`[Authorize(Roles="Teacher")]`)

Route	Purpose
`/Dashboard`	Stats + recent activity
`/Courses` `Index/Create/Edit/Delete`	Manage courses
`/Classes` `Index/Create/Edit/Delete`	Manage classes
`/Videos?classId=` `Index/Create/Edit/Delete`	Manage a class's videos
`/Quizzes?classId=` `Index/Create/Edit/Delete/Details`	Manage quizzes
`/Questions?quizId=` `Create/Edit/Delete`	Manage quiz questions
`/Exams?courseId=` `Index/Create/Edit/Delete/Details`	Manage exams
`/ExamQuestions?examId=` `Create/Edit/Delete`	Manage exam questions
`/Results/Quiz/{id}`, `/Results/Exam/{id}`	View student results
`/Revenue`	Earnings & sales history

Student (`[Authorize(Roles="Student")]`)

Route	Purpose
`/Catalog` , `/Catalog/Details/{id}`	Browse & view courses
`POST /Catalog/Enroll` , `POST /Catalog/EnrollClass`	Free enrollment (course/class)
`/Payment/Checkout?courseId=` , `/Payment/CheckoutClass?classId=`	Paid checkout
`POST /Payment/Pay` , `POST /Payment/PayClass`	Complete payment + enroll
`/Cards` `Index/Create` , `POST /Cards/Delete`	Payment cards + history
`/Learn` , `/Learn/Course/{id}` , `/Learn/Class/{id}`	Enrolled learning experience
`/Assessments/Quiz/{id}` , `/Assessments/Exam/{id}`	Take an assessment
`POST /Assessments/SubmitQuiz` , `POST /Assessments/SubmitExam`	Submit & grade

Both roles (`[Authorize]`)

Route	Purpose
`/Profile` , `/Profile/Edit` , `/Profile/Password`	View/edit profile, change password

---

7. Key Business Logic

- **Enrollment access:** a student can access a class if enrolled in its **course** OR that specific **class** (`HasClassAccess`).
- **Class-only courses:** `Course.AllowCourseEnrollment = false` hides the whole-course purchase; students buy classes individually.
- **Grading:** on submit, each answer is compared to the choice where `IsCorrect = true` ; the score + total are saved as an attempt, and per-question feedback is shown.
- **Timers:** quizzes/exams with `DurationMinutes > 0` show a countdown that auto-submits at 0.
- **Revenue split:** configured in `Helpers/PlatformSettings.cs` — `PlatformFeeRate = 0.20` (platform 20% / teacher 80%). Each `Payment` is attributed to a teacher via `TeacherID` .

---

8. Migrations & Deployment

- Schema is managed by **EF Core code-first migrations** (`Migrations/` + model snapshot).
- **Local:** `Update-Database` (Package Manager Console) or `dotnet ef database update` .
- **Deployed (MonsterASP):** the app applies pending migrations **on startup** (`Program.cs` → `db.Database.Migrate()`), since the database is only reachable from inside the host. Deploy the app and the schema is created/updated automatically, including the seeded demo teacher (`teacher@educore.local` / `Teacher@123`).

---

9. Security Notes / Limitations

- **Simulated payments** — full card number + CVV are stored for the demo only; this is **not PCI-compliant** and must not be used with real cards.
- **Video protection** — content pages are enrollment-gated, but raw video URLs remain shareable (no signed/expiring URLs).

- Passwords are hashed (PBKDF2); the seeded demo teacher should be removed before real use.